

CMCMIS_SIMPLIFIED

PHASE 12

NOTIFICATIONS + TOKEN MESSAGES

Bell in Top Navbar · Persistent Recipient-Scoped Notifications · Transient Token Capsules

FULLY SEALED · DSRC DOCUMENTATION PACK

LOCKED DELIVERABLE This document records Phase 12 as a production-grade sealed module: migrations, RBAC, backend notification emitter, frontend bell inbox, token capsule system, atomicity model, smoke verification, and handover logic.

Field	Value
Document name	phase12NotificationsTokensBellNavbarFULLYSEALED100PlusPages.docx
Project	CMCMIS_SIMPLIFIED
Phase	Phase 12 — Notifications Tokens with Bell in Top Navbar
Prepared style	DS / senior technical documentation engineer style
Source transcript	Uploaded implementation transcript: Pasted text.txt
Generated on	2026-05-20 10:56 UTC

Version, ownership, source inputs, rendering promise, and sealed status.

Document control separates the build record from chat noise. It tells future maintainers where this module starts, what is included, and which parts are intentionally locked. Phase 12 is not an experiment: it is an additive production slice that sits above the existing Phase 7–11 state machines.

DSRC-LOCK Treat this page as part of the Phase 12 sealed reference. Reopen only if Phase 12 itself is explicitly reopened.

Dimension	Locked insight
Version	v1.0 sealed
Owner lens	Senior software developer + technical documentation engineer
Scope	Notifications table, emitter, routes, bell UI, token capsule UI
Non-scope	Email notifications, SSE/websocket push, external notification channels

Two systems shipped together: persisted notifications and transient token capsules.

Phase 12 turns CMC MIS from a page-by-page workflow system into an event-aware application shell. Persistent notifications tell the correct people about durable workflow changes. Token capsules tell the current user that their local mutation was accepted or rejected. These two feedback loops are deliberately separate.

DSRC-LOCK Treat this page as part of the Phase 12 sealed reference. Reopen only if Phase 12 itself is explicitly reopened.

Dimension	Locked insight
System 1	Persistent in-app notifications
System 2	Transient token capsule messages
Verification	Phase 12 smoke matrix 14/14 green
Regression	Phase 10 smoke 49/49 and Phase 11 smoke 23/23 stayed green

Make the application talk back without breaking transactional integrity.

The goal is not simply to show a bell. The goal is to emit trusted, recipient-scoped notifications from the same transaction that changes a Job Request or Job Card. If the workflow rolls back, the notification must disappear with it. If the notification insert fails, the workflow must not silently continue.

DSRC-LOCK Treat this page as part of the Phase 12 sealed reference. Reopen only if Phase 12 itself is explicitly reopened.

Dimension	Locked insight
Mission	Atomic workflow feedback
Main UI surface	TopBar bell with unread count and dropdown
Secondary UI surface	Full /notifications inbox page
Local UX surface	2-second token capsules below navbar

The module boundaries and what changed across DB, BE, and FE.

This phase is deliberately additive. It creates one new table, two new permissions, one backend module, one frontend page, a bell dropdown, and a client-side token system. It does not mutate legacy tables destructively and does not introduce a queue broker or notification microservice.

DSRC-LOCK Treat this page as part of the Phase 12 sealed reference. Reopen only if Phase 12 itself is explicitly reopened.

Dimension	Locked insight
Database	notifications table + indexes + permissions
Backend	notifications module + emitter wiring into JR/JC services
Frontend	bell dropdown + notifications page + token capsules
Security	recipient scoping, View-Only exclusion, no cross-user leakage

What the implementation transcript confirms.

The transcript confirms migrations 430 and 431, emission wiring inside JR/JC transactions, TopBar bell replacement, /notifications page, Zustand token store, TokenHost, tokenInterceptor, auth snapshot, query-client handoff, and 14/14 smoke verification. The document expands those implementation facts into senior engineering reasoning.

DSRC-LOCK Treat this page as part of the Phase 12 sealed reference. Reopen only if Phase 12 itself is explicitly reopened.

Dimension	Locked insight
Confirmed migration	430 notifications table, 431 notification permissions
Confirmed routes	GET list/count + PATCH single/all mark-read
Confirmed tests	14/14 Phase 12 smoke, no regression
Confirmed UI	bell hidden for View-Only; tokens hidden for View-Only

Atomicity model

DECISION emit() is called inside the existing JR/JC transaction together with status history and audit rows. If notification insertion throws, the whole transaction rolls back. If the transaction rolls back for any reason, no notification row exists. This prevents false bell messages after failed workflow actions.

WHY IT MATTERS Prevents orphan notifications and makes notification data legally defensible as part of the same workflow event.

Decision field	Locked value
Decision ID	P12-D1
Topic	Atomicity model
Architectural rule	emit() is called inside the existing JR/JC transaction together with status history and audit rows. If notification insertion throws, the whole transaction rolls back. If the transaction rolls back for any reason, no notification row exists. This prevents false bell messages after failed workflow actions.
Failure mode prevented	Prevents orphan notifications and makes notification data legally defensible as part of the same workflow event.
Reopen condition	Only if Phase 12 notification semantics change globally.

Actor stripping

DECISION The acting user is always removed from the recipient list by emitter.cleanRecipients. Self-notifications are considered noise. A user who submits their own request does not need a bell message telling them they submitted it.

WHY IT MATTERS Keeps the bell meaningful and avoids notification fatigue.

Decision field	Locked value
Decision ID	P12-D2
Topic	Actor stripping
Architectural rule	The acting user is always removed from the recipient list by emitter.cleanRecipients. Self-notifications are considered noise. A user who submits their own request does not need a bell message telling them they submitted it.
Failure mode prevented	Keeps the bell meaningful and avoids notification fatigue.
Reopen condition	Only if Phase 12 notification semantics change globally.

Recipient resolution by permission

DECISION Managerial recipients are resolved by permission, not role name. `getManagerialRecipients()` resolves active users holding `job_request:approve` and uses a 60-second cache.

WHY IT MATTERS Preserves BR-RBAC-03: permission codes are the truth; role names are not hard-coded into business logic.

Decision field	Locked value
Decision ID	P12-D3
Topic	Recipient resolution by permission
Architectural rule	Managerial recipients are resolved by permission, not role name. <code>getManagerialRecipients()</code> resolves active users holding <code>job_request:approve</code> and uses a 60-second cache.
Failure mode prevented	Preserves BR-RBAC-03: permission codes are the truth; role names are not hard-coded into business logic.
Reopen condition	Only if Phase 12 notification semantics change globally.

SQL-layer recipient scope

DECISION Every read and write query filters by recipient_employee_id. A guessed notification id cannot be read or marked by another user. Cross-user mark-read returns 404 so existence is not leaked.

WHY IT MATTERS Implements defense-in-depth at the database query layer, not only at UI or service layer.

Decision field	Locked value
Decision ID	P12-D4
Topic	SQL-layer recipient scope
Architectural rule	Every read and write query filters by recipient_employee_id. A guessed notification id cannot be read or marked by another user. Cross-user mark-read returns 404 so existence is not leaked.
Failure mode prevented	Implements defense-in-depth at the database query layer, not only at UI or service layer.
Reopen condition	Only if Phase 12 notification semantics change globally.

View-Only exclusion

DECISION View-Only receives no notifications permission. The bell, /notifications page, and token capsule gate all depend on permission absence, not role-name checks.

WHY IT MATTERS Makes the exclusion auditable and future-proof if role names change.

Decision field	Locked value
Decision ID	P12-D5
Topic	View-Only exclusion
Architectural rule	View-Only receives no notifications permission. The bell, /notifications page, and token capsule gate all depend on permission absence, not role-name checks.
Failure mode prevented	Makes the exclusion auditable and future-proof if role names change.
Reopen condition	Only if Phase 12 notification semantics change globally.

Token persistence

DECISION Token capsules are not persisted. They are pure Zustand client state with no database row, audit row, or notification row. They are transient UX feedback, not workflow evidence.

WHY IT MATTERS Separates durable system events from ephemeral UI acknowledgements.

Decision field	Locked value
Decision ID	P12-D6
Topic	Token persistence
Architectural rule	Token capsules are not persisted. They are pure Zustand client state with no database row, audit row, or notification row. They are transient UX feedback, not workflow evidence.
Failure mode prevented	Separates durable system events from ephemeral UI acknowledgements.
Reopen condition	Only if Phase 12 notification semantics change globally.

Universal token wiring

DECISION A single axios response interceptor turns all successful POST/PATCH/DELETE responses into contextual success capsules and non-401 mutation failures into danger capsules.

WHY IT MATTERS Centralizes mutation feedback and prevents inconsistent per-page implementations.

Decision field	Locked value
Decision ID	P12-D7
Topic	Universal token wiring
Architectural rule	A single axios response interceptor turns all successful POST/PATCH/DELETE responses into contextual success capsules and non-401 mutation failures into danger capsules.
Failure mode prevented	Centralizes mutation feedback and prevents inconsistent per-page implementations.
Reopen condition	Only if Phase 12 notification semantics change globally.

Bell freshness

DECISION Unread count polls every 30 seconds and refetches on window focus. The token interceptor invalidates the notification query family after mutation success.

WHY IT MATTERS Gives the app a live feel without requiring SSE or websocket infrastructure in this phase.

Decision field	Locked value
Decision ID	P12-D8
Topic	Bell freshness
Architectural rule	Unread count polls every 30 seconds and refetches on window focus. The token interceptor invalidates the notification query family after mutation success.
Failure mode prevented	Gives the app a live feel without requiring SSE or websocket infrastructure in this phase.
Reopen condition	Only if Phase 12 notification semantics change globally.

ASCII architecture diagram with implementation meaning

JR/JC mutation request

↓ authenticate + authorize + validate

service opens DB transaction

↓ legacy write + status_history + audit_log

notifications.emit(conn, event_type, recipients)

↓ N × notifications.repo.insertOne(conn,row)

commit → bell count and inbox can see durable rows

ENGINEERING INTERPRETATION This diagram is not decorative. It encodes where the truth is stored, where authorization is enforced, and where failure can safely rollback. Use this page when debugging whether a missing bell row is a frontend cache issue, a backend emit issue, or a transaction rollback.

ASCII architecture diagram with implementation meaning

TopBar mounts

↓ useUnreadCount() enabled only if notifications:read-own

GET /api/v1/notifications/unread-count every 30s

↓ unread badge renders

click bell → NotificationDropdown

↓ list 10 newest + mark-read + deep-link navigation

ENGINEERING INTERPRETATION This diagram is not decorative. It encodes where the truth is stored, where authorization is enforced, and where failure can safely rollback. Use this page when debugging whether a missing bell row is a frontend cache issue, a backend emit issue, or a transaction rollback.

ASCII architecture diagram with implementation meaning

Axios mutation response

↓ tokenInterceptor filters GET/auth/notifications

2xx mutation → success capsule

4xx/5xx mutation → danger capsule

↓ Zustand tokenStore push

TokenHost renders below TopBar

↓ auto-dismiss after 2000 ms or user clicks ×

ENGINEERING INTERPRETATION This diagram is not decorative. It encodes where the truth is stored, where authorization is enforced, and where failure can safely rollback. Use this page when debugging whether a missing bell row is a frontend cache issue, a backend emit issue, or a transaction rollback.

ASCII architecture diagram with implementation meaning

```
User A guesses notification id owned by User B
↓ PATCH /notifications/:id/read
repo SQL: WHERE id=? AND recipient_employee_id=?
↓ affectedRows = 0
service returns 404
↓ no existence leak, no cross-user mutation
```

ENGINEERING INTERPRETATION This diagram is not decorative. It encodes where the truth is stored, where authorization is enforced, and where failure can safely rollback. Use this page when debugging whether a missing bell row is a frontend cache issue, a backend emit issue, or a transaction rollback.

ASCII architecture diagram with implementation meaning

```
VIEW_ONLY login
  ↓ permissions do not include notifications:read-own
TopBar hides bell
ProtectedRoute blocks /notifications
Backend authorize() rejects endpoints with 403
tokenInterceptor cannot push capsules
```

ENGINEERING INTERPRETATION This diagram is not decorative. It encodes where the truth is stored, where authorization is enforced, and where failure can safely rollback. Use this page when debugging whether a missing bell row is a frontend cache issue, a backend emit issue, or a transaction rollback.

CREATE TABLE notifications — additive and idempotent

Migration 430 creates the durable notification storage layer. It is additive only: no existing legacy table is altered. This matters because CMCMIS is running against a fixed legacy schema and every new production phase must preserve old data until deployment cutover.

Object	Purpose	Lock
notifications table	Stores one row per recipient per event	New table only
idx_notif_recipient_unread	Hot path for bell count and unread list	Composite index
idx_notif_entity	Future entity activity panels	Entity lookup
Transactional insert	Called through active service connection	No orphan rows

Colored schema dictionary with long descriptions

Column	Type	Long description
id	BIGINT UNSIGNED AUTO_INCREMENT	Primary notification id. Internal row identity. Never used as the only security boundary.
recipient_employee_id	VARCHAR(7) NOT NULL	The employee who can see this notification. Every list/count/mark SQL query filters by this column.
actor_employee_id	VARCHAR(7) NULL	The employee who caused the event. Nullable for possible future system events.
event_type	VARCHAR(60) NOT NULL	Stable uppercase event vocabulary such as JR_SUBMITTED or JC_MARKED_COMPLETE.

DSRC-DATABASE Each column has a single purpose. Recipient scoping is not a UI convention; it is encoded directly into SQL reads and updates.

Colored schema dictionary with long descriptions

Column	Type	Long description
entity_type	ENUM('JOB_REQUEST','JOB_CARD','EQUIPMENT')	Entity family used for deep-linking and future activity panels.
entity_id	VARCHAR(20) NOT NULL	Natural key converted to string: JR number, JC section job number, or equipment composite slug.
title	VARCHAR(160) NOT NULL	Short user-facing text rendered in dropdown and inbox.
body	VARCHAR(500) NULL	Longer explanatory body. Denormalized to keep bell dropdown a narrow read.

DSRC-DATABASE Each column has a single purpose. Recipient scoping is not a UI convention; it is encoded directly into SQL reads and updates.

Colored schema dictionary with long descriptions

Column	Type	Long description
deep_link	VARCHAR(200) NULL	Relative frontend route opened when the user clicks the notification.
is_read	TINYINT(1) NOT NULL DEFAULT 0	Unread/read state used by bell count and inbox filters.
created_at	DATETIME(6) DEFAULT CURRENT_TIMESTAMP(6)	Microsecond timestamp matching audit/status-history precision.
read_at	DATETIME(6) NULL	Stamped when user marks read; NULL means never read.

DSRC-DATABASE Each column has a single purpose. Recipient scoping is not a UI convention; it is encoded directly into SQL reads and updates.

(recipient_employee_id, is_read, created_at)

INDEX PURPOSE Supports unread count and newest-first list for one user. This is the hottest query in Phase 12 because the TopBar bell polls it.

Index	Columns	Query it optimizes	Reason
idx_notif_recipient_unread	(recipient_employee_id, is_read, created_at)	recipient list/count or entity activity lookup	Supports unread count and newest-first list for one user. This is the hottest query in Phase 12 because the TopBar bell polls it.

(entity_type, entity_id)

INDEX PURPOSE Supports future entity activity panels on Job Card and Job Request details. This phase does not expose that panel, but the index keeps the data model ready.

Index	Columns	Query it optimizes	Reason
idx_notif_entity	(entity_type, entity_id)	recipient list/count or entity activity lookup	Supports future entity activity panels on Job Card and Job Request details. This phase does not expose that panel, but the index keeps the data model ready.

notifications:read-own and notifications:mark-own

Migration 431 adds two explicit notification permissions. It grants both to Super Admin, Lab In-Charge, Lab Engineer, and Normal User. It intentionally grants neither to View-Only. The absence of permission is the only View-Only rule required; no role-name if-condition is needed in frontend or backend.

Permission	SA	LIC	LE	NU	VO	Long description
notifications:read-own	YES	YES	YES	YES	NO	Allows listing own notifications and polling unread count.
notifications:mark-own	YES	YES	YES	YES	NO	Allows marking own notifications read individually or in bulk.

Zod validation for list query, id param, and empty mark-all body. It rejects malformed input before repository SQL is reached, keeping the repo focused on scoped persistence.

File	Primary responsibility	Important lock
notifications.validators.js	Zod validation for list query, id param, and empty mark-all body. It rejects malformed input before repository SQL is reached, keeping the repo focused on scoped persistence.	Keep the repo/service/controller separation; future edits must not leak SQL into controllers.

The only notification module file that owns raw SQL. It implements `insertOne(conn,row)`, `listForUser`, `countUnread`, `markRead`, and `markAllRead`. Every read/write path filters by `recipient_employee_id`.

File	Primary responsibility	Important lock
<code>notifications.repo.js</code>	The only notification module file that owns raw SQL. It implements <code>insertOne(conn,row)</code> , <code>listForUser</code> , <code>countUnread</code> , <code>markRead</code> , and <code>markAllRead</code> . Every read/write path filters by <code>recipient_employee_id</code> .	Keep the repo/service/controller separation; future edits must not leak SQL into controllers.

The workflow hook. Called inside active JR/JC service transactions. It resolves event templates, dedupes recipients, strips the actor, and inserts one row per recipient using the caller connection.

File	Primary responsibility	Important lock
notifications.emitter.js	The workflow hook. Called inside active JR/JC service transactions. It resolves event templates, dedupes recipients, strips the actor, and inserts one row per recipient using the caller connection.	Keep the repo/service/controller separation; future edits must not leak SQL into controllers.

Centralized recipient resolver. It gets active employees by permission code and exposes `getManagerialRecipients()` backed by `job_request:approve` holders with a 60-second in-process cache.

File	Primary responsibility	Important lock
<code>notifications.recipients.js</code>	Centralized recipient resolver. It gets active employees by permission code and exposes <code>getManagerialRecipients()</code> backed by <code>job_request:approve</code> holders with a 60-second in-process cache.	Keep the repo/service/controller separation; future edits must not leak SQL into controllers.

Read-side facade for HTTP use. It lists notifications, counts unread, marks one read, and marks all read. Cross-user or already-read single mark returns notFound semantics.

File	Primary responsibility	Important lock
notifications.service.js	Read-side facade for HTTP use. It lists notifications, counts unread, marks one read, and marks all read. Cross-user or already-read single mark returns notFound semantics.	Keep the repo/service/controller separation; future edits must not leak SQL into controllers.

Four thin HTTP shims. Pulls employeeId from req.user and delegates to service. No business logic is hidden here.

File	Primary responsibility	Important lock
notifications.controller.js	Four thin HTTP shims. Pulls employeeId from req.user and delegates to service. No business logic is hidden here.	Keep the repo/service/controller separation; future edits must not leak SQL into controllers.

URL wiring: GET list, GET unread-count, PATCH read-all, PATCH :id/read. Each route uses authenticate and authorize with the correct notification permission.

File	Primary responsibility	Important lock
notifications.routes.js	URL wiring: GET list, GET unread-count, PATCH read-all, PATCH :id/read. Each route uses authenticate and authorize with the correct notification permission.	Keep the repo/service/controller separation; future edits must not leak SQL into controllers.

Mounts /api/v1/notifications into the existing API app without creating a separate service boundary.

File	Primary responsibility	Important lock
server.js mount	Mounts /api/v1/notifications into the existing API app without creating a separate service boundary.	Keep the repo/service/controller separation; future edits must not leak SQL into controllers.

Emitter inserted inside existing transaction boundary

Emits JR_DRAFT_SAVED or JR_SUBMITTED depending on whether submit_now is true. The emit call sits before commit, so a rollback removes both the request and the notification.

Service path	Event emitted	Atomic rule	Recipient logic
createJobRequest	JR_* event	emit(conn,...) before conn.commit()	Actor stripped; managers resolved by permission; owner added where appropriate.

Emitter inserted inside existing transaction boundary

Emits JR_SUBMITTED when an existing draft becomes submitted. Managerial recipients are notified; the actor is stripped.

Service path	Event emitted	Atomic rule	Recipient logic
submitJobRequest	JR_* event	emit(conn,...) before conn.commit()	Actor stripped; managers resolved by permission; owner added where appropriate.

Emitter inserted inside existing transaction boundary

Emits JR_EDIT_DRAFT for draft saves. This is a low-noise event because only meaningful draft-change surfaces should notify the right participants.

Service path	Event emitted	Atomic rule	Recipient logic
editDraftJobRequest	JR_* event	emit(conn,...) before conn.commit()	Actor stripped; managers resolved by permission; owner added where appropriate.

Emitter inserted inside existing transaction boundary

Emits JR_CANCELLED when a draft is logically cancelled with reason. The design ensures cancelled notifications match actual persisted status.

Service path	Event emitted	Atomic rule	Recipient logic
cancelDraftJobRequest	JR_* event	emit(conn,...) before conn.commit()	Actor stripped; managers resolved by permission; owner added where appropriate.

Emitter inserted inside existing transaction boundary

Emits JR_APPROVED_CONVERTED and JC_CREATED in the same approval/assignment/job-card creation transaction.

Service path	Event emitted	Atomic rule	Recipient logic
convertJobRequest	JR_* event	emit(conn,...) before conn.commit()	Actor stripped; managers resolved by permission; owner added where appropriate.

Emitter inserted inside existing transaction boundary

Emits JR_REJECTED with reason. The body is designed to show the rejection reason clearly in bell dropdown and inbox.

Service path	Event emitted	Atomic rule	Recipient logic
rejectJobRequest	JR_* event	emit(conn,...) before conn.commit()	Actor stripped; managers resolved by permission; owner added where appropriate.

Job Card workflow notification hook

Emits JC_TAB_UPDATED with tabHint inference so the user sees what changed without opening the card.

Service path	Event emitted	Why this event matters	Atomic rule
patchJobCardTab	JC_* event	Emits JC_TAB_UPDATED with tabHint inference so the user sees what changed without opening the card.	Same transaction as state transition and audit/status-history.

Job Card workflow notification hook

Emits JC_START_WORK when a card transitions into active work. It keeps the work queue visible.

Service path	Event emitted	Why this event matters	Atomic rule
startWorkJobCard	JC_* event	Emits JC_START_WORK when a card transitions into active work. It keeps the work queue visible.	Same transaction as state transition and audit/status-history.

Job Card workflow notification hook

Emits JC_MARKED_COMPLETE when an engineer marks the card complete and verification is needed.

Service path	Event emitted	Why this event matters	Atomic rule
markCompleteJobCard	JC_* event	Emits JC_MARKED_COMPLETE when an engineer marks the card complete and verification is needed.	Same transaction as state transition and audit/status-history.

Job Card workflow notification hook

Emits JC_VERIFIED_CLOSED and also notifies the linked JR owner so the requester knows work is closed.

Service path	Event emitted	Why this event matters	Atomic rule
verifyCloseJobCard	JC_* event	Emits JC_VERIFIED_CLOSED and also notifies the linked JR owner so the requester knows work is closed.	Same transaction as state transition and audit/status-history.

Job Card workflow notification hook

Emits JC_REOPENED with reason so assigned staff understands why rework exists.

Service path	Event emitted	Why this event matters	Atomic rule
reopenJobCard	JC_* event	Emits JC_REOPENED with reason so assigned staff understands why rework exists.	Same transaction as state transition and audit/status-history.

notifications:read-own

Returns paginated notifications for caller only. Supports unreadOnly, page, and page_size.

Endpoint	Permission	Security behavior	UX behavior
GET /api/v1/notifications	notifications:read-own	Scoped by recipient_employee_id in SQL	Returns paginated notifications for caller only. Supports unreadOnly, page, and page_size.

notifications:read-own

Returns { unread } for bell badge. Polling endpoint; narrow and indexed.

Endpoint	Permission	Security behavior	UX behavior
GET /api/v1/notifications/unread-count	notifications:read-own	Scoped by recipient_employee_id in SQL	Returns { unread } for bell badge. Polling endpoint; narrow and indexed.

notifications:mark-own

Marks one own notification as read. Cross-user ids return 404.

Endpoint	Permission	Security behavior	UX behavior
PATCH /api/v1/notifications/:id/read	notifications:mark-own	Scoped by recipient_employee_id in SQL	Marks one own notification as read. Cross-user ids return 404.

notifications:mark-own

Marks all unread caller notifications as read and returns count.

Endpoint	Permission	Security behavior	UX behavior
PATCH /api/v1/notifications/read-all	notifications:mark-own	Scoped by recipient_employee_id in SQL	Marks all unread caller notifications as read and returns count.

Bell + inbox + token capsule implementation

Axios wrappers for list, unread-count, mark-read, and mark-all-read. No UI logic exists here.

Frontend file	Responsibility	Important behavior
lib/api/notifications.js	Axios wrappers for list, unread-count, mark-read, and mark-all-read. No UI logic exists here.	Permission-aware and designed to avoid View-Only noise.

React-query hooks: `useUnreadCount`, `useNotificationList`, `useNotificationActions`, and `useCanReadNotifications`. It prevents View-Only network calls by gating before query execution.

Frontend file	Responsibility	Important behavior
<code>lib/hooks/useNotifications.js</code>	React-query hooks: <code>useUnreadCount</code> , <code>useNotificationList</code> , <code>useNotificationActions</code> , and <code>useCanReadNotifications</code> . It prevents View-Only network calls by gating before query execution.	Permission-aware and designed to avoid View-Only noise.

Bell + inbox + token capsule implementation

Bell dropdown showing 10 newest notifications, unread dot, mark-all button, and deep-link navigation.

Frontend file	Responsibility	Important behavior
components/notifications/NotificationDropdown.jsx	Bell dropdown showing 10 newest notifications, unread dot, mark-all button, and deep-link navigation.	Permission-aware and designed to avoid View-Only noise.

Bell + inbox + token capsule implementation

Full inbox with pagination, unread-only toggle, mark-all-read, and per-row deep links.

Frontend file	Responsibility	Important behavior
pages/notifications/Notifications.jsx	Full inbox with pagination, unread-only toggle, mark-all-read, and per-row deep links.	Permission-aware and designed to avoid View-Only noise.

Bell + inbox + token capsule implementation

Replaces placeholder bell with real unread count, dropdown, outside-click close, and View-Only invisibility.

Frontend file	Responsibility	Important behavior
components/TopBar.jsx	Replaces placeholder bell with real unread count, dropdown, outside-click close, and View-Only invisibility.	Permission-aware and designed to avoid View-Only noise.

Bell + inbox + token capsule implementation

Zustand store with push, dismiss, max-visible=3, and auto-dismiss=2000 ms.

Frontend file	Responsibility	Important behavior
lib/tokens/tokenStore.js	Zustand store with push, dismiss, max-visible=3, and auto-dismiss=2000 ms.	Permission-aware and designed to avoid View-Only noise.

Capsule renderer mounted in Layout between TopBar and page content. Shows message, optional subtext, timestamp, and close button.

Frontend file	Responsibility	Important behavior
components/tokens/TokenHost.jsx	Capsule renderer mounted in Layout between TopBar and page content. Shows message, optional subtext, timestamp, and close button.	Permission-aware and designed to avoid View-Only noise.

Universal axios response interceptor. Successful mutations create success capsules; non-401 mutation failures create danger capsules.

Frontend file	Responsibility	Important behavior
lib/tokens/tokenInterceptor.js	Universal axios response interceptor. Successful mutations create success capsules; non-401 mutation failures create danger capsules.	Permission-aware and designed to avoid View-Only noise.

Bell + inbox + token capsule implementation

Calls `setAuthSnapshot(user)` so the `tokenInterceptor` knows whether the current user is allowed to see capsules.

Frontend file	Responsibility	Important behavior
lib/auth-context.jsx	Calls <code>setAuthSnapshot(user)</code> so the <code>tokenInterceptor</code> knows whether the current user is allowed to see capsules.	Permission-aware and designed to avoid View-Only noise.

Hands the QueryClient to the interceptor so notification caches can be invalidated immediately after mutations.

Frontend file	Responsibility	Important behavior
main.jsx	Hands the QueryClient to the interceptor so notification caches can be invalidated immediately after mutations.	Permission-aware and designed to avoid View-Only noise.

Bell + inbox + token capsule implementation

Mounts TokenHost below TopBar and above main content.

Frontend file	Responsibility	Important behavior
components/Layout.jsx	Mounts TokenHost below TopBar and above main content.	Permission-aware and designed to avoid View-Only noise.

Bell + inbox + token capsule implementation

Adds /notifications route guarded by notifications:read-own.

Frontend file	Responsibility	Important behavior
App.jsx	Adds /notifications route guarded by notifications:read-own.	Permission-aware and designed to avoid View-Only noise.

JOB_REQUEST

Draft saved notification for request owner contexts. Low priority but useful for confirming server-side persistence.

Event type	Entity type	Deep-link thinking	Recipient thinking	Long description
JR_DRAFT_SAVED	JOB_REQUEST	Relative path to JR/JC/equipment detail	Actor stripped; role-permission-based recipients	Draft saved notification for request owner contexts. Low priority but useful for confirming server-side persistence.

JOB_REQUEST

Submitted request is visible to managerial recipients. Actor is stripped to avoid self-noise.

Event type	Entity type	Deep-link thinking	Recipient thinking	Long description
JR_SUBMITTED	JOB_REQUEST	Relative path to JR/JC/equipment detail	Actor stripped; role-permission-based recipients	Submitted request is visible to managerial recipients. Actor is stripped to avoid self-noise.

JOB_REQUEST

Draft update event; confirms that changes persisted when the user edits before submission.

Event type	Entity type	Deep-link thinking	Recipient thinking	Long description
JR_EDIT_DRAFT	JOB_REQUEST	Relative path to JR/JC/equipment detail	Actor stripped; role-permission-based recipients	Draft update event; confirms that changes persisted when the user edits before submission.

JOB_REQUEST

Approval plus auto Job Card creation. Deep link prefers the created Job Card when available.

Event type	Entity type	Deep-link thinking	Recipient thinking	Long description
JR_APPROVED_CONVERTED	JOB_REQUEST	Relative path to JR/JC/equipment detail	Actor stripped; role-permission-based recipients	Approval plus auto Job Card creation. Deep link prefers the created Job Card when available.

JOB_REQUEST

Rejection event with reason in body so the submitter has immediate actionable context.

Event type	Entity type	Deep-link thinking	Recipient thinking	Long description
JR_REJECTED	JOB_REQUEST	Relative path to JR/JC/equipment detail	Actor stripped; role-permission-based recipients	Rejection event with reason in body so the submitter has immediate actionable context.

JOB_REQUEST

Cancellation event with reason. Helps trace why a draft no longer proceeds.

Event type	Entity type	Deep-link thinking	Recipient thinking	Long description
JR_CANCELLED	JOB_REQUEST	Relative path to JR/JC/equipment detail	Actor stripped; role-permission-based recipients	Cancellation event with reason. Helps trace why a draft no longer proceeds.

JOB_CARD

New Job Card was created and assigned. Engineers and managers can jump directly to card detail.

Event type	Entity type	Deep-link thinking	Recipient thinking	Long description
JC_CREATED	JOB_CARD	Relative path to JR/JC/equipment detail	Actor stripped; role-permission-based recipients	New Job Card was created and assigned. Engineers and managers can jump directly to card detail.

JOB_CARD

Signals active work has begun. Useful for managers tracking queue movement.

Event type	Entity type	Deep-link thinking	Recipient thinking	Long description
JC_START_WORK	JOB_CARD	Relative path to JR/JC/equipment detail	Actor stripped; role-permission-based recipients	Signals active work has begun. Useful for managers tracking queue movement.

JOB_CARD

General tab-save event with tabHint so update context is not lost.

Event type	Entity type	Deep-link thinking	Recipient thinking	Long description
JC_TAB_UPDATED	JOB_CARD	Relative path to JR/JC/equipment detail	Actor stripped; role-permission-based recipients	General tab-save event with tabHint so update context is not lost.

JOB_CARD

Template reserved for child row changes. Makes maintenance/spares/task additions event-friendly.

Event type	Entity type	Deep-link thinking	Recipient thinking	Long description
JC_CHILD_ROW_ADDED	JOB_CARD	Relative path to JR/JC/equipment detail	Actor stripped; role-permission-based recipients	Template reserved for child row changes. Makes maintenance/spares/task additions event-friendly.

JOB_CARD

Completion gate event. Indicates card is ready for verification or closure action.

Event type	Entity type	Deep-link thinking	Recipient thinking	Long description
JC_MARKED_COMPLETE	JOB_CARD	Relative path to JR/JC/equipment detail	Actor stripped; role-permission-based recipients	Completion gate event. Indicates card is ready for verification or closure action.

JOB_CARD

Final close event. Also informs linked request owner that the service is done.

Event type	Entity type	Deep-link thinking	Recipient thinking	Long description
JC_VERIFIED_CLOSED	JOB_CARD	Relative path to JR/JC/equipment detail	Actor stripped; role-permission-based recipients	Final close event. Also informs linked request owner that the service is done.

JOB_CARD

Rework event with reason. Critical for audit clarity and engineer attention.

Event type	Entity type	Deep-link thinking	Recipient thinking	Long description
JC_REOPENED	JOB_CARD	Relative path to JR/JC/equipment detail	Actor stripped; role-permission-based recipients	Rework event with reason. Critical for audit clarity and engineer attention.

EQUIPMENT

Light coverage event for equipment creation, ready for expansion when equipment workflows deepen.

Event type	Entity type	Deep-link thinking	Recipient thinking	Long description
EQUIPMENT_REGISTERED	EQUIPMENT	Relative path to JR/JC/equipment detail	Actor stripped; role-permission-based recipients	Light coverage event for equipment creation, ready for expansion when equipment workflows deepen.

EQUIPMENT

Light coverage event for verification, useful for future equipment lifecycle notifications.

Event type	Entity type	Deep-link thinking	Recipient thinking	Long description
EQUIPMENT_VERIFIED	EQUIPMENT	Relative path to JR/JC/equipment detail	Actor stripped; role-permission-based recipients	Light coverage event for verification, useful for future equipment lifecycle notifications.

Token capsule UX logic

The token store is client-only, transient, and intentionally not persisted. It holds at most three visible capsules and auto-dismisses after 2000 milliseconds.

Aspect	Value
State	tokens[]
Max visible	3
Auto-dismiss	2000 ms
Persistence	none

Capsules render as rounded pills with semantic colors: success, info, and danger. Each includes a message, optional subtitle, timestamp, and close button.

Aspect	Value
Success	emerald
Info	sky
Danger	red
Position	below TopBar in Layout

Token capsule UX logic

The interceptor ignores GET/HEAD/OPTIONS, auth routes, and notifications routes. It maps POST/PATCH/DELETE successes to contextual messages and failed mutations to danger tokens.

Aspect	Value
2xx mutation	success capsule
4xx/5xx mutation	danger capsule except 401
GET	ignored
View-Only	no capsule

After successful mutations, the interceptor invalidates notification query keys so the bell badge can refresh without waiting for the next poll.

Aspect	Value
Query family	[notifications]
Trigger	mutation success
Purpose	fresh bell count
Constraint	no websocket in Phase 12

Anonymous /notifications returns 401. View-Only returns 403 for list and unread-count because permissions are absent.

Smoke check	Expected result	Interpretation
E7 anonymous and View-Only gates	PASS	Anonymous /notifications returns 401. View-Only returns 403 for list and unread-count because permissions are absent.

Acceptance proof

Submitting a Job Request inserts four notification rows in the tested DB, including managers. The actor DS00001 is stripped.

Smoke check	Expected result	Interpretation
E1 JR submit emits rows	PASS	Submitting a Job Request inserts four notification rows in the tested DB, including managers. The actor DS00001 is stripped.

Acceptance proof

An unrelated normal user such as AC06216 does not receive noise from a request they do not manage.

Smoke check	Expected result	Interpretation
E1 unrelated Normal user not notified	PASS	An unrelated normal user such as AC06216 does not receive noise from a request they do not manage.

Acceptance proof

DS00001 cannot mark SA79900 notification read. The scoped update returns 404, avoiding existence leak.

Smoke check	Expected result	Interpretation
E4 cross-user mark-read isolation	PASS	DS00001 cannot mark SA79900 notification read. The scoped update returns 404, avoiding existence leak.

Acceptance proof

Every returned notification row belongs to the caller. No foreign recipient rows appear.

Smoke check	Expected result	Interpretation
E4 list scope cleanliness	PASS	Every returned notification row belongs to the caller. No foreign recipient rows appear.

Acceptance proof

mark-read flips unread count and stamps read_at.

Smoke check	Expected result	Interpretation
E5 mark-read state change	PASS	mark-read flips unread count and stamps read_at.

Acceptance proof

A rejected/failed transition leaves notification counts unchanged.

Smoke check	Expected result	Interpretation
E8 atomicity check	PASS	A rejected/failed transition leaves notification counts unchanged.

Acceptance proof

No legacy SQL column names leak into service/controller/routes/validators/recipients. Raw SQL remains in repo layer.

Smoke check	Expected result	Interpretation
E9 repo aliasing	PASS	No legacy SQL column names leak into service/controller/routes/validators/recipients. Raw SQL remains in repo layer.

Acceptance proof

Phase 10 smoke stays 49/49 green after Phase 12.

Smoke check	Expected result	Interpretation
Phase 10 regression	PASS	Phase 10 smoke stays 49/49 green after Phase 12.

Acceptance proof

Phase 11 smoke stays 23/23 green after Phase 12.

Smoke check	Expected result	Interpretation
Phase 11 regression	PASS	Phase 11 smoke stays 23/23 green after Phase 12.

LOGIC DEVELOPMENT Start with the invariant. If the notification must represent a real workflow event, it must live inside the same database transaction as the event.

Thinking layer	Question to ask	Phase 12 answer
How to think: DB first	Where should truth live?	Start with the invariant. If the notification must represent a real workflow event, it must live inside the same database transaction as the event.

LOGIC DEVELOPMENT A future admin may rename roles or split roles. Permission codes survive those changes. Therefore notification access is gated by notification permissions, not role strings.

Thinking layer	Question to ask	Phase 12 answer
How to think: permissions before roles	Where should truth live?	A future admin may rename roles or split roles. Permission codes survive those changes. Therefore notification access is gated by notification permissions, not role strings.

LOGIC DEVELOPMENT A service-layer if-check can be bypassed by a future method. SQL WHERE recipient_employee_id=? is the hard gate.

Thinking layer	Question to ask	Phase 12 answer
How to think: scoped SQL	Where should truth live?	A service-layer if-check can be bypassed by a future method. SQL WHERE recipient_employee_id=? is the hard gate.

LOGIC DEVELOPMENT Recipient resolver caching improves performance but does not define correctness. Correctness comes from permission grants and SQL filters.

Thinking layer	Question to ask	Phase 12 answer
How to think: cache as optimization	Where should truth live?	Recipient resolver caching improves performance but does not define correctness. Correctness comes from permission grants and SQL filters.

LOGIC DEVELOPMENT Notifications are durable and workflow-related. Tokens are transient and mutation-feedback-related. Never mix the two stores.

Thinking layer	Question to ask	Phase 12 answer
How to think: durable vs transient	Where should truth live?	Notifications are durable and workflow-related. Tokens are transient and mutation-feedback-related. Never mix the two stores.

LOGIC DEVELOPMENT TopBar is the correct bell location because every authenticated page uses Layout. TokenHost belongs below TopBar because it is global but should not obscure navigation.

Thinking layer	Question to ask	Phase 12 answer
How to think: frontend shell integration	Where should truth live?	TopBar is the correct bell location because every authenticated page uses Layout. TokenHost belongs below TopBar because it is global but should not obscure navigation.

LOGIC DEVELOPMENT Actor stripping and unrelated-user exclusion are as important as emitting events. A noisy bell becomes ignored.

Thinking layer	Question to ask	Phase 12 answer
How to think: avoiding notification fatigue	Where should truth live?	Actor stripping and unrelated-user exclusion are as important as emitting events. A noisy bell becomes ignored.

LOGIC DEVELOPMENT A failed mutation must not write a notification. Always test the count before and after a deliberately failed transition.

Thinking layer	Question to ask	Phase 12 answer
How to think: rollback testing	Where should truth live?	A failed mutation must not write a notification. Always test the count before and after a deliberately failed transition.

LOGIC DEVELOPMENT This design can later add SSE or WebSocket by pushing invalidation events. The database model does not need to change.

Thinking layer	Question to ask	Phase 12 answer
How to think: future SSE	Where should truth live?	This design can later add SSE or WebSocket by pushing invalidation events. The database model does not need to change.

LOGIC DEVELOPMENT Notifications are not audit logs. Audit is legal history; notification is user awareness. Both point to the same event but serve different readers.

Thinking layer	Question to ask	Phase 12 answer
How to think: audit relationship	Where should truth live?	Notifications are not audit logs. Audit is legal history; notification is user awareness. Both point to the same event but serve different readers.

Light code only — implementation memory aid

```
CREATE TABLE IF NOT EXISTS notifications (  
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  recipient_employee_id VARCHAR(7) NOT NULL,  
  event_type VARCHAR(60) NOT NULL,  
  entity_type ENUM('JOB_REQUEST', 'JOB_CARD', 'EQUIPMENT') NOT NULL,  
  entity_id VARCHAR(20) NOT NULL,  
  title VARCHAR(160) NOT NULL,  
  is_read TINYINT(1) NOT NULL DEFAULT 0  
);
```

CODE NOTE This snippet is illustrative and intentionally short. Full source remains in the project repository; this document focuses on architectural understanding.

Light code only — implementation memory aid

```
KEY idx_notif_recipient_unread  
  (recipient_employee_id, is_read, created_at)
```

CODE NOTE This snippet is illustrative and intentionally short. Full source remains in the project repository; this document focuses on architectural understanding.

Light code only — implementation memory aid

```
SELECT id, title, body, deep_link, is_read
FROM notifications
WHERE recipient_employee_id = ?
ORDER BY created_at DESC, id DESC
LIMIT ? OFFSET ?;
```

CODE NOTE This snippet is illustrative and intentionally short. Full source remains in the project repository; this document focuses on architectural understanding.

Light code only — implementation memory aid

UPDATE notifications

```
SET is_read = 1, read_at = COALESCE(read_at, NOW(6))  
WHERE id = ? AND recipient_employee_id = ? AND is_read = 0;
```

CODE NOTE This snippet is illustrative and intentionally short. Full source remains in the project repository; this document focuses on architectural understanding.

Light code only — implementation memory aid

```
await emit({
  conn, event_type: 'JR_SUBMITTED', entity_type: 'JOB_REQUEST',
  entity_id: jrNo, entity, actor, recipients
});
```

CODE NOTE This snippet is illustrative and intentionally short. Full source remains in the project repository; this document focuses on architectural understanding.

Light code only — implementation memory aid

```
pushToken({  
  message: 'Job Request submitted',  
  variant: 'success'  
});
```

CODE NOTE This snippet is illustrative and intentionally short. Full source remains in the project repository; this document focuses on architectural understanding.

Light code only — implementation memory aid

```
useQuery({
  queryKey: ['notifications','unread-count'],
  queryFn: ({ signal }) => fetchUnreadCount(signal),
  refetchInterval: 30000,
  enabled: canRead
});
```

CODE NOTE This snippet is illustrative and intentionally short. Full source remains in the project repository; this document focuses on architectural understanding.

Light code only — implementation memory aid

```
<TopBar />  
<TokenHost />  
<main>{children}</main>
```

CODE NOTE This snippet is illustrative and intentionally short. Full source remains in the project repository; this document focuses on architectural understanding.

notifications.validators.js

Zod validation for list query, id param, and empty mark-all body. It rejects malformed input before repository SQL is reached, keeping the repo focused on scoped persistence.

File	Layer	Long description	Change type
notifications.validators.js	Backend	Zod validation for list query, id param, and empty mark-all body. It rejects malformed input before repository SQL is reached, keeping the repo focused on scoped persistence.	New or edited in Phase 12

notifications.repo.js

The only notification module file that owns raw SQL. It implements insertOne(conn,row), listForUser, countUnread, markRead, and markAllRead. Every read/write path filters by recipient_employee_id.

File	Layer	Long description	Change type
notifications.repo.js	Backend	The only notification module file that owns raw SQL. It implements insertOne(conn,row), listForUser, countUnread, markRead, and markAllRead. Every read/write path filters by recipient_employee_id.	New or edited in Phase 12

notifications.emitter.js

The workflow hook. Called inside active JR/JC service transactions. It resolves event templates, dedupes recipients, strips the actor, and inserts one row per recipient using the caller connection.

File	Layer	Long description	Change type
notifications.emitter.js	Backend	The workflow hook. Called inside active JR/JC service transactions. It resolves event templates, dedupes recipients, strips the actor, and inserts one row per recipient using the caller connection.	New or edited in Phase 12

notifications.recipients.js

Centralized recipient resolver. It gets active employees by permission code and exposes getManagerialRecipients() backed by job_request:approve holders with a 60-second in-process cache.

File	Layer	Long description	Change type
notifications.recipients.js	Backend	Centralized recipient resolver. It gets active employees by permission code and exposes getManagerialRecipients() backed by job_request:approve holders with a 60-second in-process cache.	New or edited in Phase 12

notifications.service.js

Read-side facade for HTTP use. It lists notifications, counts unread, marks one read, and marks all read. Cross-user or already-read single mark returns notFound semantics.

File	Layer	Long description	Change type
notifications.service.js	Backend	Read-side facade for HTTP use. It lists notifications, counts unread, marks one read, and marks all read. Cross-user or already-read single mark returns notFound semantics.	New or edited in Phase 12

notifications.controller.js

Four thin HTTP shims. Pulls employeeId from req.user and delegates to service. No business logic is hidden here.

File	Layer	Long description	Change type
notifications.controller.js	Backend	Four thin HTTP shims. Pulls employeeId from req.user and delegates to service. No business logic is hidden here.	New or edited in Phase 12

notifications.routes.js

URL wiring: GET list, GET unread-count, PATCH read-all, PATCH :id/read. Each route uses authenticate and authorize with the correct notification permission.

File	Layer	Long description	Change type
notifications.routes.js	Backend	URL wiring: GET list, GET unread-count, PATCH read-all, PATCH :id/read. Each route uses authenticate and authorize with the correct notification permission.	New or edited in Phase 12

server.js mount

Mounts /api/v1/notifications into the existing API app without creating a separate service boundary.

File	Layer	Long description	Change type
server.js mount	Backend	Mounts /api/v1/notifications into the existing API app without creating a separate service boundary.	New or edited in Phase 12

lib/api/notifications.js

Axios wrappers for list, unread-count, mark-read, and mark-all-read. No UI logic exists here.

File	Layer	Long description	Change type
lib/api/notifications.js	Frontend	Axios wrappers for list, unread-count, mark-read, and mark-all-read. No UI logic exists here.	New or edited in Phase 12

lib/hooks/useNotifications.js

React-query hooks: useUnreadCount, useNotificationList, useNotificationActions, and useCanReadNotifications. It prevents View-Only network calls by gating before query execution.

File	Layer	Long description	Change type
lib/hooks/useNotifications.js	Frontend	React-query hooks: useUnreadCount, useNotificationList, useNotificationActions, and useCanReadNotifications. It prevents View-Only network calls by gating before query execution.	New or edited in Phase 12

components/notifications/NotificationDropdown.jsx

Bell dropdown showing 10 newest notifications, unread dot, mark-all button, and deep-link navigation.

File	Layer	Long description	Change type
components/notifications/NotificationDropdown.jsx	Frontend	Bell dropdown showing 10 newest notifications, unread dot, mark-all button, and deep-link navigation.	New or edited in Phase 12

pages/notifications/Notifications.jsx

Full inbox with pagination, unread-only toggle, mark-all-read, and per-row deep links.

File	Layer	Long description	Change type
pages/notifications/Notifications.jsx	Frontend	Full inbox with pagination, unread-only toggle, mark-all-read, and per-row deep links.	New or edited in Phase 12

components/TopBar.jsx

Replaces placeholder bell with real unread count, dropdown, outside-click close, and View-Only invisibility.

File	Layer	Long description	Change type
components/TopBar.jsx	Frontend	Replaces placeholder bell with real unread count, dropdown, outside-click close, and View-Only invisibility.	New or edited in Phase 12

lib/tokens/tokenStore.js

Zustand store with push, dismiss, max-visible=3, and auto-dismiss=2000 ms.

File	Layer	Long description	Change type
lib/tokens/tokenStore.js	Frontend	Zustand store with push, dismiss, max-visible=3, and auto-dismiss=2000 ms.	New or edited in Phase 12

components/tokens/TokenHost.jsx

Capsule renderer mounted in Layout between TopBar and page content. Shows message, optional subtext, timestamp, and close button.

File	Layer	Long description	Change type
components/tokens/TokenHost.jsx	Frontend	Capsule renderer mounted in Layout between TopBar and page content. Shows message, optional subtext, timestamp, and close button.	New or edited in Phase 12

lib/tokens/tokenInterceptor.js

Universal axios response interceptor. Successful mutations create success capsules; non-401 mutation failures create danger capsules.

File	Layer	Long description	Change type
lib/tokens/tokenInterceptor.js	Frontend	Universal axios response interceptor. Successful mutations create success capsules; non-401 mutation failures create danger capsules.	New or edited in Phase 12

lib/auth-context.jsx

Calls setAuthSnapshot(user) so the tokenInterceptor knows whether the current user is allowed to see capsules.

File	Layer	Long description	Change type
lib/auth-context.jsx	Frontend	Calls setAuthSnapshot(user) so the tokenInterceptor knows whether the current user is allowed to see capsules.	New or edited in Phase 12

main.jsx

Hands the QueryClient to the interceptor so notification caches can be invalidated immediately after mutations.

File	Layer	Long description	Change type
main.jsx	Frontend	Hands the QueryClient to the interceptor so notification caches can be invalidated immediately after mutations.	New or edited in Phase 12

components/Layout.jsx

Mounts TokenHost below TopBar and above main content.

File	Layer	Long description	Change type
components/Layout.jsx	Frontend	Mounts TokenHost below TopBar and above main content.	New or edited in Phase 12

App.jsx

Adds /notifications route guarded by notifications:read-own.

File	Layer	Long description	Change type
App.jsx	Frontend	Adds /notifications route guarded by notifications:read-own.	New or edited in Phase 12

PHASE 12 · PAGE 129 · ACCEPTANCE CHECKLIST

Final DS lock section

FINAL SEAL DB migrations applied, permissions verified, View-Only excluded, routes mounted, emitter wired into JR/JC, bell UI live, tokens live, smoke green.

Checklist item	Status	Evidence
Acceptance Checklist	LOCKED	DB migrations applied, permissions verified, View-Only excluded, routes mounted, emitter wired into JR/JC, bell UI live, tokens live, smoke green.
Smoke matrix	14/14 green	Acceptance verified
Regression	Phase 10 49/49 + Phase 11 23/23	No breakage

FINAL SEAL No cross-user read, no cross-user mark-read, 404 existence hiding, SQL recipient filter, permission-based View-Only exclusion.

Checklist item	Status	Evidence
Security Checklist	LOCKED	No cross-user read, no cross-user mark-read, 404 existence hiding, SQL recipient filter, permission-based View-Only exclusion.
Smoke matrix	14/14 green	Acceptance verified
Regression	Phase 10 49/49 + Phase 11 23/23	No breakage

FINAL SEAL Bell count uses composite index, recipient resolver uses 60-second cache, notification list is paginated, token capsules are in-memory only.

Checklist item	Status	Evidence
Performance Checklist	LOCKED	Bell count uses composite index, recipient resolver uses 60-second cache, notification list is paginated, token capsules are in-memory only.
Smoke matrix	14/14 green	Acceptance verified
Regression	Phase 10 49/49 + Phase 11 23/23	No breakage

FINAL SEAL Repo owns SQL, service owns behavior, controller is thin, routes declare auth/permission, FE uses hooks and API wrappers.

Checklist item	Status	Evidence
Maintainability Checklist	LOCKED	Repo owns SQL, service owns behavior, controller is thin, routes declare auth/permission, FE uses hooks and API wrappers.
Smoke matrix	14/14 green	Acceptance verified
Regression	Phase 10 49/49 + Phase 11 23/23	No breakage

FINAL SEAL Phase 12 is ready for app-level usage. Future SSE/email can attach to the same emitted events without changing the core table.

Checklist item	Status	Evidence
Production Readiness Notes	LOCKED	Phase 12 is ready for app-level usage. Future SSE/email can attach to the same emitted events without changing the core table.
Smoke matrix	14/14 green	Acceptance verified
Regression	Phase 10 49/49 + Phase 11 23/23	No breakage

FINAL SEAL Phase 12 Notifications + Token Messages is sealed. Treat this document as the handover record unless the phase is explicitly reopened.

Checklist item	Status	Evidence
Final Locked Seal	LOCKED	Phase 12 Notifications + Token Messages is sealed. Treat this document as the handover record unless the phase is explicitly reopened.
Smoke matrix	14/14 green	Acceptance verified
Regression	Phase 10 49/49 + Phase 11 23/23	No breakage

Titles should be short, bodies should explain action, deep links should go to the actionable entity.

Topic	Long description	Operational rule	Risk if ignored
Notification UX copy standards	Titles should be short, bodies should explain action, deep links should go to the actionable entity.	Follow sealed Phase 12 pattern	Noise, broken RBAC, or orphan rows

Newest 10 rows, unread tint, mark all read, close button, outside-click handling, deep-link navigation.

Topic	Long description	Operational rule	Risk if ignored
Bell dropdown behavior	Newest 10 rows, unread tint, mark all read, close button, outside-click handling, deep-link navigation.	Follow sealed Phase 12 pattern	Noise, broken RBAC, or orphan rows

Paginated list, unread-only toggle, full timestamps, actor id display, row click navigation.

Topic	Long description	Operational rule	Risk if ignored
Inbox behavior	Paginated list, unread-only toggle, full timestamps, actor id display, row click navigation.	Follow sealed Phase 12 pattern	Noise, broken RBAC, or orphan rows

Mutation failures become red capsules except 401. Auth/session handling remains separate.

Topic	Long description	Operational rule	Risk if ignored
Token danger variant	Mutation failures become red capsules except 401. Auth/session handling remains separate.	Follow sealed Phase 12 pattern	Noise, broken RBAC, or orphan rows

All successful mutations receive short contextual confirmation.

Topic	Long description	Operational rule	Risk if ignored
Token success variant	All successful mutations receive short contextual confirmation.	Follow sealed Phase 12 pattern	Noise, broken RBAC, or orphan rows

The interceptor maps known routes to messages and falls back to Created/Saved/Removed.

Topic	Long description	Operational rule	Risk if ignored
Mutation URL inference	The interceptor maps known routes to messages and falls back to Created/Saved/Removed.	Follow sealed Phase 12 pattern	Noise, broken RBAC, or orphan rows

No email, no SMS, no background queue, no websocket, no activity panel, no stored tokens.

Topic	Long description	Operational rule	Risk if ignored
Non-goals	No email, no SMS, no background queue, no websocket, no activity panel, no stored tokens.	Follow sealed Phase 12 pattern	Noise, broken RBAC, or orphan rows

Email can subscribe to the same event vocabulary but should not replace in-app notifications.

Topic	Long description	Operational rule	Risk if ignored
Future extension: email	Email can subscribe to the same event vocabulary but should not replace in-app notifications.	Follow sealed Phase 12 pattern	Noise, broken RBAC, or orphan rows

SSE can replace polling by pushing cache invalidation messages to the browser.

Topic	Long description	Operational rule	Risk if ignored
Future extension: SSE	SSE can replace polling by pushing cache invalidation messages to the browser.	Follow sealed Phase 12 pattern	Noise, broken RBAC, or orphan rows

The idx_notif_entity index already prepares the database for per-entity activity panes.

Topic	Long description	Operational rule	Risk if ignored
Future extension: entity activity panel	The idx_notif_entity index already prepares the database for per-entity activity panes.	Follow sealed Phase 12 pattern	Noise, broken RBAC, or orphan rows

Smoke tests validate behavior, not just compilation. They test auth, RBAC, scoping, read state, and atomicity.

Topic	Long description	Operational rule	Risk if ignored
Testing philosophy	Smoke tests validate behavior, not just compilation. They test auth, RBAC, scoping, read state, and atomicity.	Follow sealed Phase 12 pattern	Noise, broken RBAC, or orphan rows

No migration is destructive. To rollback, drop notifications table and remove notification permissions.

Topic	Long description	Operational rule	Risk if ignored
Rollback philosophy	No migration is destructive. To rollback, drop notifications table and remove notification permissions.	Follow sealed Phase 12 pattern	Noise, broken RBAC, or orphan rows

Long-form color table page

Every page stores a reusable decision, not merely a chat transcript.

Topic	Long description	Operational rule	Risk if ignored
Documentation philosophy	Every page stores a reusable decision, not merely a chat transcript.	Follow sealed Phase 12 pattern	Noise, broken RBAC, or orphan rows

Do not redesign when extending; clone the sealed pattern and add only what the next phase requires.

Topic	Long description	Operational rule	Risk if ignored
DS locked doctrine	Do not redesign when extending; clone the sealed pattern and add only what the next phase requires.	Follow sealed Phase 12 pattern	Noise, broken RBAC, or orphan rows

PHASE 12 · PAGE 149 · ONE-PAGE FINAL SUMMARY

Phase 12 sealed in one compact reference

Phase 12 adds durable in-app notifications and transient token capsules. Notifications are inserted inside JR/JC transactions, scoped by recipient at SQL level, gated by notification permissions, and surfaced through a real TopBar bell plus a full inbox page. Tokens are client-only Zustand capsules, auto-dismiss after two seconds, and are fired centrally by an axios mutation interceptor. View-Only sees neither notifications nor tokens. Smoke matrix is 14/14 green, with Phase 10 and Phase 11 regression matrices staying green.

Area	Final status
Database	notifications table + indexes + permissions created
Backend	notifications module and emitter wired into JR/JC service transactions
Frontend	Bell dropdown, full inbox page, token capsules, query invalidation
Security	Recipient scoped SQL, View-Only excluded, 404 existence hiding
Verification	14/14 Phase 12 smoke; no Phase 10/11 regression

SEALED Total authored pages with explicit page breaks: 149. This DOCX is the Phase 12 sealed documentation deliverable.